

In [58]: *#Inicio importando las librerias que necesito*

```
import pandas as pd
import numpy as np
import matplotlib as plt
import matplotlib.pyplot as plt
from pathlib import Path
```

In [59]: *#Carga del dataset de Sabadell en Python, se agrego La Libreria*

```
##from pathlib import Path para que suba sin necesidad de colocar el directorio
# en el df_datasabadell se demora uno aproximadamente 4 seg porque es de 10000 regi
input_dir = Path.cwd() / "input"
xlsx_files = []
if input_dir.exists():
    xlsx_files = sorted(input_dir.glob("*.xlsx"))
else:
    xlsx_files = sorted(Path.cwd().glob("*.xlsx"))

if not xlsx_files:
    raise FileNotFoundError(
        f"No se encontró ningún archivo .xlsx en {input_dir} ni en el directorio ac
    )

excel_path = xlsx_files[0]
df_datasabadell = pd.read_excel(excel_path, engine="openpyxl")
print(f"Listo! Excel cargado desde: {excel_path}")
print(f"output_dir será: {Path.cwd() / 'output'}")
```

Listo! Excel cargado desde: c:\Users\jmendozarojas\OneDrive - Getronics\Escritorio\INESDI\TFM\Entrega 1 -TFM Ethos\BD_Sabadell.xlsx
output_dir será: c:\Users\jmendozarojas\OneDrive - Getronics\Escritorio\INESDI\TFM\Entrega 1 -TFM Ethos\output

In [61]: *#Realizamos La confirmación de la carga del df_datasabadell*

```
df_datasabadell.head()
```

Out[61]:

	id_cliente	antig	edad	totalpasta	impoped	impopeh	ind_cuentaexpansion	ind_cuent
--	------------	-------	------	------------	---------	---------	---------------------	-----------

0	1	194	84	52043.01	5516.79	23100.89	1
1	2	244	75	4219.46	744.74	0.00	0
2	3	336	84	55585.45	1152.96	400.20	0
3	4	278	55	5813.22	11449.97	15317.10	0
4	5	36	55	19000.00	0.00	0.00	0

5 rows × 83 columns



```
In [62]: #funcion para identificar los campos nulos de la tabla
print("Nulos por columna:")
print(df_datosabadell.isnull().sum())
```

```
Nulos por columna:
id_cliente      0
antig           0
edad           0
totalpasta      0
impoped         0
...
rdom            0
tpv            0
margecomercialut 0
total_proteccio 0
fase          114
Length: 83, dtype: int64
```

```
In [63]: #Consultamos la tabla
df_datosabadell.head()
```

```
Out[63]:
```

	id_cliente	antig	edad	totalpasta	impoped	impopeh	ind_cuentaexpansion	ind_cuenta
0	1	194	84	52043.01	5516.79	23100.89		1
1	2	244	75	4219.46	744.74	0.00		0
2	3	336	84	55585.45	1152.96	400.20		0
3	4	278	55	5813.22	11449.97	15317.10		0
4	5	36	55	19000.00	0.00	0.00		0

5 rows × 83 columns



```
In [64]: #Función para contar la cantidad total de clientes en la BD
total_clientes=df_datosabadell.id_cliente.count()
print(f"El total de clientes registrados en la BD es : {total_clientes}")
```

El total de clientes registrados en la BD es : 10000

```
In [65]: #Perfilados estadísticos de los diferentes campos, se muestran valores
#mínimos y máximos.
# Análisis:
# Edad =0.000000 corresponde a padres que crean una cuenta de ahorro para sus hijos
# totalpasta =0.000000 corresponde a los clientes que no tienen fondos.
# impoped = 0.000000 corresponde a los clientes que no tuvieron egresos en su cuenta
resumen = df_datosabadell.describe()
print(resumen)
```

	id_cliente	antig	edad	totalpasta	impoped \
count	10000.00000	10000.000000	10000.000000	1.000000e+04	1.000000e+04
mean	5000.50000	218.333500	49.354400	2.384789e+04	4.945997e+03
std	2886.89568	795.911927	20.248531	8.650505e+04	2.296940e+04
min	1.00000	1.000000	0.000000	0.000000e+00	0.000000e+00
25%	2500.75000	47.000000	36.000000	2.874675e+02	7.624000e+01
50%	5000.50000	121.000000	49.000000	2.123225e+03	1.690965e+03
75%	7500.25000	233.000000	64.000000	1.707842e+04	4.574812e+03
max	10000.00000	9999.000000	116.000000	4.798534e+06	1.055250e+06

	impopeh	ind_cuentaexpansion	ind_cuentaexpansionnegocios \
count	1.000000e+04	10000.000000	10000.000000
mean	5.433100e+03	0.347200	0.007000
std	2.168653e+04	0.476104	0.083377
min	0.000000e+00	0.000000	0.000000
25%	4.888250e+01	0.000000	0.000000
50%	2.414000e+03	0.000000	0.000000
75%	5.740130e+03	1.000000	0.000000
max	1.064506e+06	1.000000	1.000000

	ind_cuentaexpansionpremium	ind_cuentaexperiencia	...	prov \
count	10000.000000	10000.000000	...	10000.000000
mean	0.022800	0.080400	...	26.041600
std	0.149273	0.271925	...	25.616645
min	0.000000	0.000000	...	1.000000
25%	0.000000	0.000000	...	8.000000
50%	0.000000	0.000000	...	20.000000
75%	0.000000	0.000000	...	33.000000
max	1.000000	1.000000	...	99.000000

	credit	revolving	tothipo	nominaimp	rdom \
count	10000.000000	10000.000000	1.000000e+04	10000.000000	10000.000000
mean	100.536738	53.858220	1.361625e+04	1141.936248	8.978100
std	2457.438350	332.113919	5.211624e+04	4359.877628	11.710924
min	0.000000	0.000000	0.000000e+00	0.000000	0.000000
25%	0.000000	0.000000	0.000000e+00	0.000000	0.000000
50%	0.000000	0.000000	0.000000e+00	0.000000	5.000000
75%	0.000000	0.000000	0.000000e+00	0.000000	14.000000
max	134200.660000	7901.990000	2.538921e+06	335186.920000	86.000000

	tpv	margecomercialut	total_proteccio	fase
count	10000.00000	10000.000000	10000.000000	9886.000000
mean	0.00070	128.151498	0.223400	19.240036
std	0.02645	462.907653	0.591208	9.887232
min	0.00000	-1961.800000	0.000000	0.000000
25%	0.00000	6.077500	0.000000	11.000000
50%	0.00000	36.050000	0.000000	22.000000
75%	0.00000	138.195000	0.000000	28.000000
max	1.00000	31639.730000	6.000000	30.000000

[8 rows x 81 columns]

```
In [66]: #Analisis de los datos atipicos, conteno e identificación
cantidad_9999 = (df_datasabadell['antig'] == 9999).sum()
print(f"Cantidad de registros con antig == '9999, posibles datos atipicos': {cantidad_9999}")
```

Cantidad de registros con antig == '9999, posibles datos atipicos': 64

```
In [67]: #Se tomo la decisión de eliminar Los 64 registros con esos valores antig=9999, de a
df_delete = df_datosabadell[df_datosabadell['antig'] == 9999].copy()
print(f"CORRECCIÓN: Registros con antig == 9999 (para eliminar): {len(df_delete)}")
```

CORRECCIÓN: Registros con antig == 9999 (para eliminar): 64

```
In [68]: df_delete.head()
```

```
Out[68]:
```

	id_cliente	antig	edad	totalpasta	impoped	impopeh	ind_cuentaexpansion	ind_cu
1474	1475	9999	60	0.00	205.49	0.0	0	
1600	1601	9999	37	0.00	495.02	0.0	0	
2304	2305	9999	40	6428.52	0.00	0.0	0	
2711	2712	9999	57	0.00	0.00	0.0	0	
2737	2738	9999	45	0.00	427.58	0.0	0	

5 rows × 83 columns



```
In [69]: # Para eliminar realmente del DataFrame principal
df_datosabadell = df_datosabadell[df_datosabadell['antig'] != 9999]

print("\nEliminados los registros con valores 9999")
```

Eliminados los registros con valores 9999

```
In [70]: #Validamos la eliminacion de Los datos atipicos y en total queda 9936 clientes
total_clientes=df_datosabadell.id_cliente.count()
print(f"El total de clientes registrados en la BD es : {total_clientes}")
```

El total de clientes registrados en la BD es : 9936

```
In [72]: #Segmentacion de clientes por tipo de cuenta
# Lista de columnas a seleccionar
columnas_seleccion = [
    'id_cliente',
    'ind_cuentaexpansion',
    'ind_cuentaexpansionnegocios',
    'ind_cuentaexpansionpremium',
    'ind_cuentaexperiencia',
    'ind_cuentaprimera',
    'ind_cuentaproyeccion',
    'ind_cvautonomos',
    'ind_cvdirect',
    'ind_cvhabit',
    'ind_cvjove',
    'ind_cvjunior',
    'ind_cvmas',
    'ind_cvprestige',
    'ind_cvsenior',
    'ind_estalinv',
```

```

        'ind_estalvi',

    ]

    # Seleccionar las columnas (syntax correcta)
    df_tipocuenta = df_datosabadell[columnas_seleccion].copy()

```

In [37]: df_tipocuenta.head()

Out[37]:

	id_cliente	ind_cuentaexpansion	ind_cuentaexpansionnegocios	ind_cuentaexpansionpremi
0	1	1	0	
1	2	0	0	
2	3	0	0	
3	4	0	0	
4	5	0	0	

```

In [ ]: # Función para realizar group by por tipo de cuenta y contar cuántas personas tiene
def group_by_tipo_cuenta(df, account_columns):

    # Verificar que las columnas existan en el DataFrame
    missing_columns = [col for col in account_columns if col not in df.columns]
    if missing_columns:
        print(f"Advertencia: Las siguientes columnas no existen en el DataFrame: {missing_columns}")
        # Remover columnas faltantes de la lista
        account_columns = [col for col in account_columns if col not in missing_columns]

    # Crear un DataFrame para almacenar los resultados
    results = []

    for account_type in account_columns:
        # Contar cuántas personas tienen esta cuenta (valor = 1)
        count = df[df[account_type] == 1][account_type].count()

        # Calcular porcentaje del total
        total_clients = df['id_cliente'].nunique() # Usar nunique para contar clientes únicos
        percentage = (count / total_clients) * 100 if total_clients > 0 else 0

        results.append({
            'Tipo de Cuenta': account_type,
            'Cantidad': count,
            'Porcentaje': f"{percentage:.2f}%"
        })

    # Crear DataFrame de resultados
    results_df = pd.DataFrame(results)

    # Ordenar por cantidad de personas de mayor a menor
    results_df = results_df.sort_values(by='Cantidad', ascending=False)

    return results_df

```

```
# También necesitas definir columnas_cuenta (sin 'id_cliente')
columnas_cuenta = [
    'ind_cuentaexpansion',
    'ind_cuentaexpansionnegocios',
    'ind_cuentaexpansionpremium',
    'ind_cuentaexperiencia',
    'ind_cuentaprimera',
    'ind_cuentaproyeccion',
    'ind_cvautonomos',
    'ind_cvdirect',
    'ind_cvhabit',
    'ind_cvjove',
    'ind_cvjunior',
    'ind_cvmas',
    'ind_cvprestige',
    'ind_cvsenior',
    'ind_estalinv',
    'ind_estalvi',
]

```

```
In [74]: # Usar la función con df_datosabadell en lugar de df_tipocuenta
# IMPORTANTE: Primero el DataFrame, luego las columnas
resultados = group_by_tipo_cuenta(df_datosabadell, columnas_cuenta)
print(resultados)
```

Advertencia: Las siguientes columnas no existen en el DataFrame: ['ind_estrangers']

	Tipo de Cuenta	Cantidad	Porcentaje
0	ind_cuentaexpansion	3472	34.94%
14	ind_estalinv	3337	33.58%
15	ind_estalvi	3061	30.81%
3	ind_cuentaexperiencia	804	8.09%
5	ind_cuentaproyeccion	660	6.64%
9	ind_cvjove	660	6.64%
10	ind_cvjunior	541	5.44%
4	ind_cuentaprimera	541	5.44%
12	ind_cvprestige	355	3.57%
2	ind_cuentaexpansionpremium	228	2.29%
7	ind_cvdirect	138	1.39%
13	ind_cvsenior	80	0.81%
1	ind_cuentaexpansionnegocios	70	0.70%
11	ind_cvmas	35	0.35%
6	ind_cvautonomos	11	0.11%
8	ind_cvhabit	2	0.02%

```
In [75]: #Se calcula el promedio de antigüedad del cliente, coincide con el POWER BI
promedio_antigüedad = (df_datosabadell['antig'].mean())
print(f"El promedio de antigüedad de los clientes es: {promedio_antigüedad}")
```

El promedio de antigüedad de los clientes es: 155.33403784219

```
In [76]: #Se calcula el min de antigüedad del cliente coincide con el POWER BI
minimo_antigüedad = (df_datosabadell['antig'].min())
print(f"El Mínimo de antigüedad de la BD de clientes es de: {minimo_antigüedad}")
```

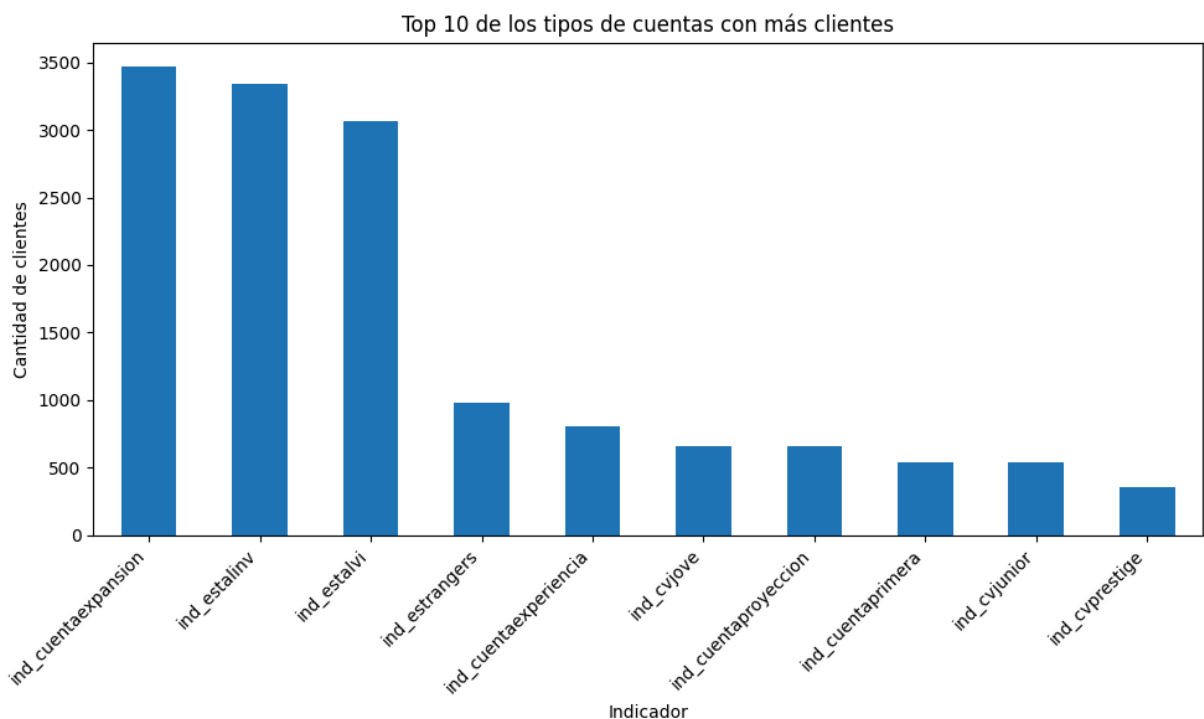
El Mínimo de antigüedad de la BD de clientes es de: 1

```
In [40]: # Contar cuántos clientes tienen cada indicador activo
conteo_indicadores = {} # <- DEFINIR EL DICCIONARIO PRIMERO

for col in columnas_seleccion:
    if col != 'id_cliente' and col in df_tipocuenta.columns:
        # Asumiendo que los indicadores son binarios (0/1 o True/False)
        conteo = df_tipocuenta[col].sum() if df_tipocuenta[col].dtype in [int, float] else df_tipocuenta[col].sum()
        conteo_indicadores[col] = conteo # <- Ahora sí está definido
print(conteo_indicadores) # Mostrar todos los conteos, no solo el último
```

```
{'ind_cuentaexpansion': np.int64(3472), 'ind_cuentaexpansionnegocios': np.int64(70),
'ind_cuentaexpansionpremium': np.int64(228), 'ind_cuentaexperiencia': np.int64(804),
'ind_cuentaprimera': np.int64(541), 'ind_cuentaproyeccion': np.int64(660), 'ind_cvautonomos': np.int64(11),
'ind_cvdirect': np.int64(138), 'ind_cvhabit': np.int64(2), 'ind_cvjove': np.int64(660), 'ind_cvjunior': np.int64(541),
'ind_cvmass': np.int64(35), 'ind_cvprestige': np.int64(355), 'ind_cvsenior': np.int64(80), 'ind_estalinv': np.int64(3337),
'ind_estalvi': np.int64(3061), 'ind_estrangers': np.int64(984)}
```

```
In [41]: # Crear gráfico (funcionará ahora que conteo_indicadores está definido)
pd.Series(conteo_indicadores).sort_values(ascending=False).head(10).plot(
    kind='bar',
    title='Top 10 de los tipos de cuentas con más clientes',
    figsize=(10, 6)
)
plt.xlabel('Indicador')
plt.ylabel('Cantidad de clientes')
plt.xticks(rotation=45, ha='right')
plt.tight_layout()
plt.show()
```



```
In [77]: # Crear carpeta BD_Actualizada si no existe
bd_actualizada_dir = Path.cwd() / "Output_BD_Actualizada"
bd_actualizada_dir.mkdir(exist_ok=True)
```

```
output_file = bd_actualizada_dir / f"datasabadell_actualizado.xlsx"

# Exportar el DataFrame actualizado
df_datasabadell.to_excel(output_file, index=False)
print(f"\nDataFrame exportado exitosamente a: {output_file}")
print(f"Total de registros exportados: {len(df_datasabadell)}")
```

DataFrame exportado exitosamente a: c:\Users\jmendozarojas\OneDrive - Getronics\Escritorio\INESDI\TFM\Entrega 1 -TFM Ethos\Output_BD_Actualizada\datasabadell_actualizado.xlsx

Total de registros exportados: 9936